

100 Days to Become a Technically Exceptional Head of AI

Vision

This is **not** a Python course.

This is a structured 100-day transformation from an experienced AI leader who primarily reviews and directs technical work into a leader who can confidently prototype, review, architect, debug, and deliver AI systems end to end.

The objective is **not** to become a full-time software engineer.

The objective is to become the kind of Head of AI who earns technical credibility from engineering teams, gains confidence from CTOs, and can independently transform an idea into a working proof of concept.

Python is the vehicle—not the destination.

Why This Journey Exists

Over the past decade, analytics leadership gradually shifted:

SQL → SAS → Python → Machine Learning → LLMs → AI Agents.

While leading teams, I focused on architecture, delivery, stakeholder management, strategy, model governance, and AI initiatives. My teams increasingly adopted Python while my own day-to-day work became less hands-on.

When generative AI arrived, writing code became easier, but technical fluency became even more important.

AI can generate code.

It cannot consistently judge:

- whether the architecture is correct,
- whether the business logic is complete,
- whether the implementation is maintainable,
- whether the design will scale,
- or whether the solution aligns with business objectives.

The goal of these 100 days is therefore **technical fluency**, not memorizing syntax.

Desired Outcome

At the end of these 100 days I should be able to:

- Read production Python code comfortably.
 - Build AI proof-of-concepts independently.
 - Review code written by engineers or AI assistants.
 - Design AI systems from ideation through deployment.
 - Build APIs, RAG systems, and AI agents.
 - Explain architectural decisions confidently.
 - Lead AI engineering teams with hands-on credibility.
-

Guiding Principles

1. Learn for Leadership

Every topic should answer one question:

How does this make me a stronger AI leader?

If a topic has no practical leadership value, it is lower priority.

2. Build, Don't Memorize

The objective is never to memorize syntax.

The objective is to understand how software is designed.

3. Business First

Every coding exercise should represent a business scenario.

Examples:

- Banking
- Fraud Detection
- AML
- Customer Analytics
- Risk Assessment
- AI Applications

Toy problems exist only to teach concepts.

Real business problems develop engineering judgment.

4. Code Every Day

Consistency beats intensity.

Even 90 focused minutes every day for 100 days will produce dramatically better results than occasional long study sessions.

5. AI is a Coding Partner

AI is encouraged.

However:

Never copy code without understanding it.

Every line committed should be explainable.

The objective is to become better at evaluating AI-generated code.

Learning Philosophy

Programming consists of four activities.

1. Understanding the problem.
2. Clarifying business assumptions.
3. Designing the solution.
4. Writing code.

Typing code is only one part of software engineering.

Phase 1 – Python Fluency (Days 1–20)

Objective

Develop Python fluency until writing Python feels as natural as writing SQL.

Topics

- Variables
- Data Types
- Strings
- Lists
- Dictionaries
- Tuples
- Sets
- Conditionals
- Loops
- Functions
- Modules
- Files
- Error Handling

Projects

- Banking Console Applications
- Salary Analytics
- Customer Risk Assessment
- Fraud Detection Exercises

Outcome

Python syntax becomes automatic.

Phase 2 – Professional Python (Days 21–40)

Objective

Write production-quality Python.

Topics

- Object-Oriented Programming

- Dataclasses
- Packages
- Logging
- Testing
- Type Hints
- Virtual Environments
- Code Formatting
- Debugging

Projects

- Banking Domain Model
- Fraud Detection Engine
- Modular Python Applications

Outcome

Ability to review and write maintainable production code.

Phase 3 – Data Engineering & Analytics (Days 41–55)

Objective

Bridge SQL expertise into Python.

Topics

- NumPy
- Pandas
- Data Cleaning
- Feature Engineering
- Exploratory Analysis

Projects

- AML Analytics
- Customer Segmentation
- Risk Dashboards

Outcome

Perform analytics in Python as comfortably as SQL.

Phase 4 – Backend Engineering (Days 56–65)

Objective

Expose AI models through APIs.

Topics

- FastAPI
- REST APIs
- Validation
- Authentication
- Docker
- Environment Variables

Projects

- Customer Risk API
- Fraud Scoring API
- Model Inference API

Outcome

Build deployable AI services.

Phase 5 – LLM Engineering (Days 66–80)

Objective

Understand modern AI implementation.

Topics

- Transformers
- Tokenization
- Embeddings
- Prompt Engineering
- OpenAI SDK
- Structured Outputs
- Function Calling

Projects

- Compliance Assistant
- Document Summarizer
- Policy Chatbot

Outcome

Build production-ready LLM applications.

Phase 6 – Retrieval Augmented Generation (Days 81–87)

Topics

- Chunking
- Embeddings
- Vector Databases
- FAISS
- Hybrid Search
- Evaluation

Projects

- Banking Knowledge Assistant
- AML Policy Search
- Internal Compliance Chatbot

Outcome

Design and evaluate enterprise RAG systems.

Phase 7 – AI Agents (Days 88–94)

Topics

- LangGraph
- Multi-Agent Systems
- Tool Calling
- MCP
- Planning
- Agent Memory

Projects

- Fraud Investigation Agent
- Risk Analyst Agent
- AI Meeting Assistant

Outcome

Understand agentic AI beyond demonstrations.

Phase 8 – Executive AI Portfolio (Days 95–100)

Objective

Replace coding exercises with complete AI products.

Projects

1. AI Banking Copilot
2. Enterprise RAG Platform
3. Multi-Agent Risk Assessment System
4. Executive AI Demonstration Platform

Outcome

A portfolio demonstrating technical leadership rather than programming exercises.

Weekly Structure

Monday

New concepts

Tuesday

Coding exercises

Wednesday

Refactoring and debugging

Thursday

Reading production code

Friday

Mini project

Saturday

Documentation and technical writing

Sunday

Reflection, GitHub updates, and review

GitHub Strategy

The repository is not intended to impress recruiters through beginner exercises.

It should demonstrate continuous technical growth.

Repository Structure

README.md

ROADMAP.md

Day01/

Day02/

...

Projects/

Notes/

Reflections/

Every commit should have a meaningful message.

Every week should include a short written reflection.

By Day 100 the repository should tell the story of technical progression rather than simply contain code.

Success Criteria

Success is **not** measured by solving 100 coding problems.

Success is measured by answering "yes" to these questions:

Can I prototype an AI solution myself?

Can I confidently review Python code?

Can I mentor Python engineers technically?

Can I challenge architectural decisions?

Can I build APIs, RAG systems, and AI agents?

Can I demonstrate technical credibility during executive interviews?

If the answer to those questions is yes, the journey has succeeded.

Final Goal

The goal is not to become the best Python programmer.

The goal is to become one of the few AI leaders who combines:

- Strategic thinking
- Business understanding
- AI expertise
- Software engineering discipline
- Hands-on implementation capability

That combination creates technical credibility.

Technical credibility builds trust.

Trust creates leadership opportunities.

Leadership opportunities create career growth.

Python is simply the first step in that transformation.